

SOFTWARE TESTING SEMINAR

API Testing & Contract Testing

Nhóm 03 — SEBros

THÀNH VIÊN

Ngô Nguyễn Thế Khoa
23127065

Mạch Quốc Tấn
23127115

Ân Tiến Nguyễn An
23127148

Nguyễn Tuấn Anh
23127152

Nguyễn Lê Hồ Anh Khoa
23127211

Postman

Newman

Pact

GitHub Actions

Nội dung trình bày

- 01 Giới thiệu & Mục tiêu
- 03 Công cụ: Postman & REST Client
- 05 Automation với Newman & CI/CD
- 07 Pact: Consumer → Provider → Broker
- 09 AI hỗ trợ kiểm thử
- 02 API Testing — Khái niệm & Kỹ thuật
- 04 Demo: API Testing thực hành
- 06 Contract Testing — Vấn đề & Giải pháp
- 08 Demo: Contract Testing & Breaking Change
- 10 Tổng kết & Q&A

01 · INTRODUCTION

Giới thiệu & Mục tiêu

Seminar đi từ bối cảnh microservices đến API Testing, Contract Testing và tự động hóa CI/CD, rồi chốt bằng AI-assisted testing.

BỐI CẢNH

Microservices làm API trở thành điểm nối sống còn

MỤC TIÊU

Hiểu API, Pact và cách đưa test vào CI/CD

PHẠM VI

Product Service, Postman, Newman, GitHub Actions

PHẦN A

API Testing

Đi từ khái niệm nền tảng đến cách thiết kế test case, rồi chuyển sang công cụ và demo thực hành trên Product Service.

NỀN TẢNG

HTTP, status code, auth và request/response

THIẾT KẾ TEST

EP, BVA, DDT, Decision Table, Pairwise

CÔNG CỤ

Postman, REST Client và Product Service

API là gì?

API (Application Programming Interface): giao diện lập trình cho phép các hệ thống giao tiếp.

REST API: dùng HTTP protocol — method, URL, headers, body.

Request: **Method** + **URL** + **Headers** + **Body**

Response: **Status Code** + **Headers** + **Body**

HTTP Methods

Method	Ý nghĩa	Ví dụ
GET	Đọc dữ liệu	GET /products
POST	Tạo mới	POST /products
PUT	Cập nhật toàn bộ	PUT /product/10
PATCH	Cập nhật một phần	PATCH /product/10
DELETE	Xóa	DELETE /product/10

Nguyên tắc thiết kế REST

Statelessness

Server không lưu trạng thái phiên làm việc; mỗi request phải tự mang đầy đủ thông tin xác thực và dữ liệu.

Safety & Idempotency

GET chỉ đọc (safe). PUT, DELETE, GET có thể gọi lặp lại nhiều lần mà không thay đổi tác động phụ (idempotent).

Đọc kết quả API bằng status code

2xx

Success

200 OK, 201 Created, 204 No Content

4xx

Client Error

400, 401, 403, 404, 409, 422, 429

5xx

Server Error

500, 502, 503, 504

1xx và 3xx vẫn thuộc chuẩn HTTP, nhưng khi trình bày API Testing thì nhóm tập trung vào 2xx, 4xx và 5xx vì đây là nhóm xuất hiện nhiều nhất trong test thực tế.

¹ Tip — Khi assert API, nên kiểm tra cả status code, body schema và error message, không chỉ nhìn vào `200`.

API có Authentication vs Authorization

AuthN vs AuthZ

- **Authentication (AuthN)**: xác minh danh tính người dùng
- **Authorization (AuthZ)**: kiểm tra quyền hạn của danh tính đó
- **No Auth** chỉ phù hợp với endpoint công khai, ít nhạy cảm

Nhận diện nhanh

AuthN

Hỏi "bạn là ai?" trước khi vào hệ thống

AuthZ

Hỏi "bạn được phép làm gì?" sau khi đã biết danh tính

Product Service

Mọi endpoint đều yêu cầu `Authorization: Bearer ISO-8601 timestamp`.

```
Authorization: Bearer 2026-07-15T10:00:00.000Z
```

Token hợp lệ trong vòng 1 giờ và không ở tương lai → sai format hoặc hết hạn sẽ trả `401`.

¹ **Bearer ISO-8601** — Token dạng timestamp theo chuẩn ISO-8601 (vd: 2026-07-15T10:00:00.000Z), hợp lệ trong 1 giờ và dùng như custom bearer token trong

Các cơ chế xác thực phổ biến

BASIC AUTH

Base64 `username:password`, đơn giản nhưng chỉ nên dùng khi có HTTPS.

API KEY

Khóa tĩnh cho client, thường đặt ở header để tránh lộ trong URL.

BEARER JWT

Token 3 phần `Header.Payload.Signature`, phù hợp cho auth stateless.

OAUTH 2.0

Cơ chế ủy quyền cho app thứ ba, phổ biến với Authorization Code và Client Credentials.

MTLS

Client và server đều có chứng chỉ, phù hợp machine-to-machine hoặc microservices nội bộ.

CUSTOM TIMESTAMP

Bearer ISO-8601 của Product Service, dùng như cơ chế demo cho toàn bộ endpoint.

Trong seminar này: Product Service dùng custom bearer timestamp, nên mọi endpoint đều cần `Authorization` header hợp lệ.

Các loại Test Case cho API

Loại	Mô tả	Ví dụ
Happy Path ¹	Request hợp lệ → response đúng	GET /product/10 + valid token → 200
Negative	Input sai → xử lý lỗi đúng	GET /product/99999 → 404
Authentication	Thiếu/sai/hết hạn token → 401	Không gửi Authorization header
Validation	Body thiếu field → 400	POST thiếu "name" → 400
Schema	Response đúng cấu trúc kỳ vọng	Body có đủ id, type, name, version
Boundary	Giá trị biên	Token đúng 1 giờ trước (vừa hết hạn)

Bộ 6 loại này là nền tảng để bao phủ happy path, error path và dữ liệu biên trước khi đi sang các kỹ thuật nâng cao.

Các kỹ thuật nâng cao như EP, BVA, Decision Table, Pairwise, State Transition, API Chaining và OWASP API Security sẽ đi ở slide kế tiếp.

¹ Happy Path — Kịch bản với dữ liệu hợp lệ, đi qua luồng thành công.

Kỹ thuật nâng cao để mở rộng coverage

INPUT SPACE

EP + BVA + DDT

Dùng khi dữ liệu đầu vào có miền giá trị, có biên hoặc cần chạy nhiều bộ data.

DECISION LOGIC

Decision Table

Dùng khi nhiều điều kiện kết hợp thành nhiều kết quả, như checkout hoặc khuyến mãi.

COMBINATION

Pairwise

Giảm số lượng test mà vẫn bao phủ các cặp giá trị quan trọng.

WORKFLOW & SECURITY

State · Chaining · OWASP

Dùng cho luồng nhiều bước, tài nguyên có trạng thái, và kiểm thử rủi ro bảo mật API.

Chọn kỹ thuật theo rủi ro: dữ liệu biên, tổ hợp điều kiện, chuỗi hành vi, hay lỗi bảo mật.

Postman — Tổng quan

Postman là gì

- Nền tảng kiểm thử API **phổ biến nhất**
- Hỗ trợ Desktop App, Web và **VS Code Extension**
- Miễn phí cho cá nhân & nhóm nhỏ, nhưng có quota giới hạn ở cloud features
- Phù hợp khi cần request, scripts, runner và automation trong cùng một nơi

Các khái niệm chính

- **Collection** — Gom nhóm request
- **Environment** — Biến môi trường
- **Variable** — Giá trị động dùng lại
- **Pre-request Script** — Chạy trước request
- **Test Script** — Assertion chạy sau response

Postman không chỉ là nơi gửi request thủ công. Điểm mạnh thật sự nằm ở collection, script và khả năng kéo test sang automation sau này.

¹ **Assertion** — Câu lệnh kiểm tra tự động (vd: status = 200) chạy sau mỗi request.

Postman — Tính năng nâng cao

Collection Runner

Chạy chuỗi request theo vòng lặp hoặc theo data file.

Data-driven

Dùng CSV/JSON để mở rộng coverage mà không nhân bản script.

Newman

CLI runner để đẩy collection vào CI/CD.

Monitor

Test định kỳ tự động để quan sát API theo thời gian.

Mock Server

Giả lập API nhanh khi chưa có backend thật.

Giá trị chính

Postman trở thành workspace cho test, script và handoff sang automation.

Postman — Tổ chức Collection

Product Service — Data Driven Tests

- ├ _Setup (Pre-flight) ← sinh token
- ├ GET — Happy Path ← 4 iterations
- ├ GET — Negative ← 7 iterations
- ├ POST — Happy Path ← 2 iterations
- ├ POST — Negative ← 5 iterations
- ├ PUT — Happy Path ← 2 iterations
- ├ PUT — Negative ← 4 iterations
- ├ DELETE — Happy Path ← 1 iteration
- └ DELETE — Negative ← 4 iterations

29

Test cases

5

Endpoints

9

Folders

Postman — Script & Assertion

Pre-request Script (Collection level)

Tự động sinh Bearer token hợp lệ mỗi iteration:

- "{validToken}" → Bearer hợp lệ
- "Bearer 2020-..." → token hết hạn (negative)
- "" → xóa header (no token case)

Test Script (ví dụ)

```
pm.test("Status is 200", () => {  
    pm.response.to.have.status(200);  
});  
  
pm.test("Response has required fields", () => {  
    const json = pm.response.json();  
    pm.expect(json).to.have.property("id");  
    pm.expect(json).to.have.property("name");  
    pm.expect(json).to.have.property("type");  
});
```

Data-driven Testing

Khái niệm: Chạy cùng 1 request với **nhiều bộ dữ liệu** khác nhau. Data file: JSON hoặc CSV. Mỗi dòng = 1 iteration.

```
{
  "tc_id": "GET_ID_01",
  "description": "Existing product with valid token",
  "product_id": "10",
  "auth_header": "{{validToken}}",
  "expected_status": 200,
  "expect_field_id": "10",
  "expect_field_name": "28 Degrees"
}
```

Tách logic khỏi data

Script không đổi, chỉ thêm data

Dễ thêm case mới

Thêm dòng vào data file

Phù hợp automation

Chạy với Newman CI/CD

VS Code REST Client

REST Client (extension)

- File `.http` — viết request ngay trong VS Code
- Không cần mở Postman GUI
- Hỗ trợ biến, chaining, và assertion cơ bản

Mẫu `.http` viết request trực tiếp trong VS Code, hỗ trợ biến và request chaining.

```
@baseUrl = http://localhost:8080
@token = {{$datetime iso8601 -1 m}}

### GET /products → 200
GET {{baseUrl}}/products
Authorization: Bearer {{token}}
```

So sánh nhanh

Tiêu chí	Postman	REST Client
GUI	Đầy đủ (App/Web/Extension)	Tối giản
Data-driven	✓	✗
Script	JS đầy đủ	Hạn chế
CI/CD	Export → Newman	✗
Tiện lợi	Có App + Extension	Ngay trong editor

API mẫu: Product Service

Thông tin

- **Node.js + Express** (Port **8080**)
- Auth: **Bearer ISO-8601** (trong 1 giờ)
- Data: **In-memory** (reset khi restart)

Product Schema

```
{ "id": "10", "type": "CREDIT_CARD", "name": "28  
Degrees", "version": "v1" }
```

Ý nghĩa của service này

Mục tiêu demo

Dùng một API CRUD đơn giản để minh họa test chức năng, auth và contract.

Tính ổn định

In-memory data giúp test chạy lặp lại và không phụ thuộc trạng thái ngoài ý muốn.

Kết nối sang phần sau

Toàn bộ endpoint ở trang kế tiếp đều yêu cầu cùng một kiểu Authorization header.

Endpoint của Product Service

Endpoints

Method	Path	Success	Error
GET	/products	200 + array	401
GET	/product/:id	200 + object	401, 404
POST	/products	201 + created	400, 401
PUT	/product/:id	200 + updated	401, 404
DELETE	/product/:id	204	401, 404

Điểm cần nhớ

Authorization

Mọi endpoint đều yêu cầu `Authorization: Bearer ISO-8601 timestamp` , kể cả GET.

Coverage

Collection thực tế: 29 test cases · 5 endpoints · 9 folders

Demo bridge

Đây là API mẫu được dùng xuyên suốt cả phần API Testing lẫn Pact demo.

PHẦN B

Automation & CI/CD

Biến test thành pipeline tự động để Newman và Pact có thể chạy như một quality gate thay vì chỉ là bước kiểm tra thủ công.

NEWMAN

Chạy collection từ CLI

GITHUB ACTIONS

Tự động hóa khi push / PR

PACT GATE

Chặn breaking change trước deploy

Newman — CLI Runner

Newman là gì

- Command-line runner¹ cho **Postman Collection**
- Chạy test **không cần mở Postman GUI**
- Xuất báo cáo: **CLI, HTML, JSON**

Lệnh cốt lõi

```
newman run collection.json \  
  -e environment.json \  
  --reporters cli,html,json \  
  --reporter-html,json-reporter report.html \  
  --reporter-json-reporter report.json
```

Luồng tự động hóa

1. Khởi động Provider tại `localhost:8080`
2. Readiness probe²: gọi `/health`
3. Chạy Newman với collection + environment
4. Xuất báo cáo HTML + JSON
5. Upload artifact³ (kể cả khi test fail)

¹ **CLI runner** — Công cụ chạy test trực tiếp từ dòng lệnh, không cần giao diện.

² **readiness probe** — Request kiểm tra service đã khởi động sẵn sàng (vd: /health).

³ **artifact** — File đầu ra (report, log) được CI lưu lại để xem sau.

GitHub Actions — Newman Pipeline

Workflow: `newman-api-test.yml` | Trigger: `push/PR vào main + manual dispatch`

Checkout → Setup Node 20 → Install deps → Start Provider
→ Wait `/health` (30s timeout) → Install Newman
→ Run Newman → Upload report artifact

permissions: contents: read

Giới hạn quyền tối thiểu

timeout-minutes: 10

Tránh pipeline treo

concurrency: cancel-in-progress

Hủy run cũ khi push mới

if: always()

Luôn upload report để debug

GitHub Actions — Pact Pipeline

Workflow: `pact-verification.yml`

Consumer Pact

Sinh pact.json
npm run test:pact

Provider Verification

Verify pact thật
Chạy với API thật

can-i-deploy¹

Quality gate²
Chặn nếu incompatible

Job 1 — Consumer

Chạy consumer test → sinh pact JSON →
upload artifact → publish lên Broker

Job 2 — Provider

Download pact → chạy verifier với API thật →
publish kết quả

Job 3 — can-i-deploy

Kiểm tra compatibility matrix → chặn deploy
nếu chưa tương thích

¹ **can-i-deploy** — Lệnh của Pact Broker: kiểm tra phiên bản này có an toàn để deploy.

² **quality gate** — Cổng kiểm tra tự động — chặn pipeline nếu không đạt.

Pact Verification: local artifact vs Broker

Cách 1: Local artifact

Provider verification có thể chạy bằng artifact local qua `PACT_FILE`.

Phù hợp khi chạy lab, demo hoặc CI artifact handoff.

Cách 2: Broker / Pactflow

Khi nâng cấp lên Broker/Pactflow thì dùng `PACT_BROKER_URL` + token để publish và verify xuyên CI.

Phù hợp khi nhiều team cần compatibility matrix và deployment gate thật.

Repo hiện hỗ trợ cả hai đường đi. Nhờ vậy nhóm có thể demo bằng artifact cục bộ trước, rồi mở rộng sang mô hình Broker khi muốn scale quy trình.

Giá trị của Automation

Hai lớp bảo vệ

Lớp	Công cụ	Phát hiện
Functional	Newman/Postman	Lỗi chức năng, validation, auth, payload
Compatibility	Pact	Breaking change tại biên Consumer–Provider

Phản hồi sớm

Chạy trên mỗi push/PR

Tái lập

Log + artifact lưu lại để debug

Giảm thủ công

Không cần kiểm tra manual

Bằng chứng

Reviewer truy ngược version, test result

PHẦN C

Contract Testing

Giải quyết rủi ro breaking change tại biên Consumer–Provider và tạo ra một quy trình xác minh tương thích rõ ràng hơn.

VẤN ĐỀ

Hai service đều xanh nhưng hệ thống vẫn gãy

GIẢI PHÁP

Consumer-Driven Contract Testing

PACT

Contract, Provider State và Broker

Khi mỗi service đều "xanh"... hệ thống vẫn có thể "đỏ"

Consumer¹

Kỳ vọng field `product.name`, nhưng
Provider đổi thành `displayName`

Provider²

Unit test vẫn pass — logic và schema
mới đều đúng theo góc nhìn backend

Runtime

Lỗi chỉ lộ khi tích hợp — trên staging
hoặc production

"Hai phía có còn hiểu cùng một giao thức hay không?"

Unit test kiểm tra logic nội bộ — không kiểm chứng giả định xuyên biên giới

¹ **Consumer** — Bên gọi API (vd: FrontendWebsite).

² **Provider** — Bên cung cấp API (vd: ProductService).

Contract Testing là gì?

*Contract¹ là đặc tả **có thể thực thi** về những request Consumer gửi và response Provider cam kết đáp ứng.*

- **Request:** method, path, query, headers, body
- **Response:** status, headers, schema và matching rules
- **Context:** provider state² — điều kiện trước của interaction³

INTERACTION

Given product 10 exists
When GET /product/10
Then 200 + Product schema

Contract Testing xác minh **tính tương thích** — không chứng minh toàn bộ nghiệp vụ đúng.

¹ **Contract** — Bản cam kết bằng máy về cách hai bên giao tiếp.

² **Provider state** — Điều kiện dữ liệu cần có trước khi chạy interaction (vd: "product 10 exists").

³ **Interaction** — Một cặp request-response cụ thể trong contract.

So sánh các lớp kiểm thử

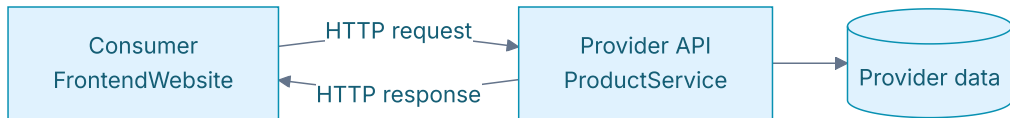
Tiêu chí	API Testing	Contract Testing	Integration ¹	E2E ²
Câu hỏi	Endpoint đúng?	Còn tương thích?	Phối hợp đúng?	Journey chạy?
Phạm vi	1+ endpoint	1 cặp C-P	Nhóm thành phần	Toàn hệ thống
Môi trường	API thật/mock	Cô lập, local/CI	Bán tích hợp	Gần production
Feedback	Nhanh-TB	Nhanh, rõ	Trung bình	Chậm
Điểm mạnh	Chức năng, auth	Chống breaking change	Wiring	User journey
Điểm mù	Phụ thuộc consumer	Business logic	Ngoài scope	Flaky, đắt

Contract Test **bổ sung** Unit / Integration / E2E — không thay thế tất cả.

¹ **Integration** — Kiểm thử nhiều thành phần phối hợp với nhau.

² **E2E** — Kiểm thử toàn bộ hành trình người dùng qua nhiều service.

Mô hình Consumer-Provider



Consumer (FrontendWebsite)

Mô tả chính xác phần API nó sử dụng → sinh Pact file (JSON)

Provider (ProductService)

Chứng minh implementation đáp ứng mọi interaction → chạy verification

Pact Broker:¹ Lưu contract + version + kết quả verification → Compatibility matrix² → `can-i-deploy` gate

¹ **Pact Broker** — Kho trung tâm lưu contract + kết quả verification.

² **Compatibility matrix** — Bảng tra cứu phiên bản Consumer nào tương thích phiên bản Provider nào.

Consumer-Driven: Vì sao?

Provider-Driven

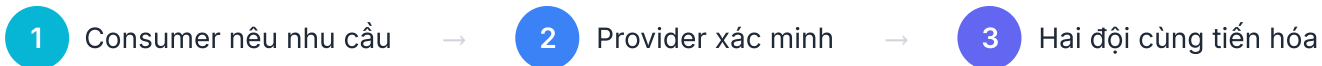
Provider công bố toàn bộ schema. Consumer phải thích nghi.

Vấn đề: Provider khó biết phần nào đang được sử dụng.

Consumer-Driven

Mỗi Consumer phát hành interaction mình phụ thuộc.
Provider xác minh tập hợp nhu cầu thực tế.

Lợi ích: Provider biết field nào đang dùng → tránh xóa/sửa nhầm.



Giới hạn của Contract Testing

Business logic nội bộ

Response đúng schema nhưng tính sai tổng tiền vẫn pass

Hạ tầng production

DNS, TLS, gateway, timeout cần lớp test/monitor khác

Hành trình nhiều service

Một pact chỉ chứng minh một ranh giới

Chất lượng API design

Tương thích ≠ dễ dùng, nhất quán, an toàn

Unit → logic · **Contract** → compatibility · **Integration** → wiring · **E2E** → critical journeys

PHẦN D

Demo Pact

Đi từ Consumer tạo contract, Provider verify contract, rồi quan sát cách một breaking change bị chặn trước khi chạm production.

CONSUMER

Sinh pact từ test thật

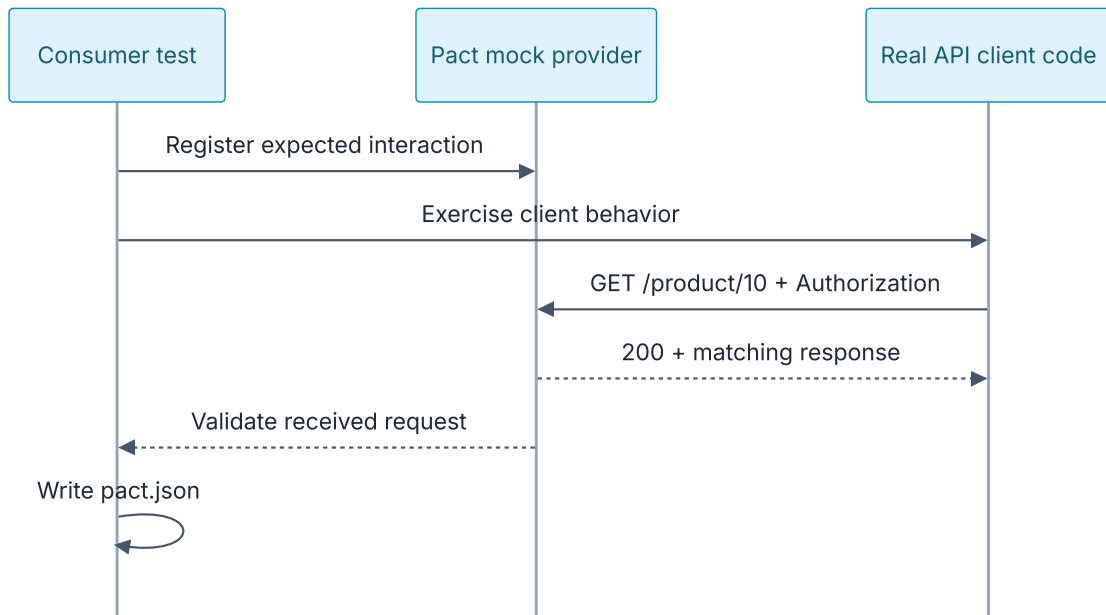
PROVIDER

Replay và verify interaction

BROKER

Lưu version và compatibility matrix

Bước 1 — Consumer tạo contract



Mock Provider¹ không thay thế code Consumer — test phải gọi **API client thật** của Consumer.

¹ **Mock provider** — Server giả do Pact dựng lên để Consumer test chạy mà không cần Provider thật.

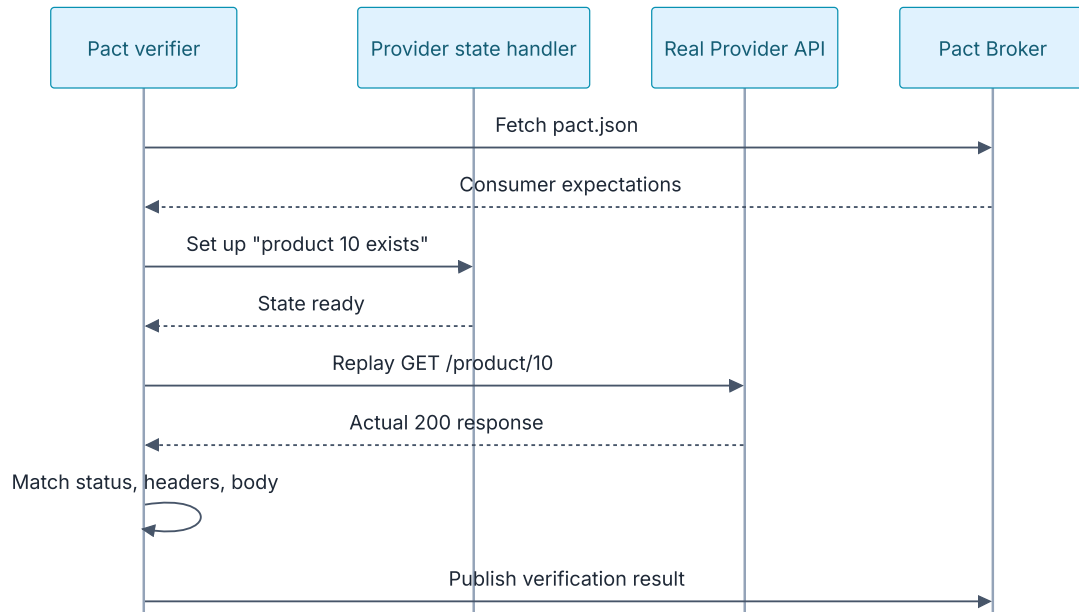
Pact JSON — Cấu trúc Interaction

```
{
  "description": "get product 10",
  "providerState": "product 10 exists",
  "request": {
    "method": "GET",
    "path": "/product/10"
  },
  "response": {
    "status": 200,
    "body": {
      "id": "10",
      "name": "28 Degrees",
      "type": "CREDIT_CARD"
    }
  },
  "matchingRules": {
    "$.body.id": "type:string",
    "$.body.name": "type:string"
  }
}
```

Nguyên tắc Matcher: Strict với điều Consumer phụ thuộc · Linh hoạt với dữ liệu động · Không over/under-specify

Repo: 10 interactions · 10 Authorization regex matchers (Bearer ISO-8601)

Bước 2 — Provider xác minh contract¹



Real routing

Request đi vào API thật

Controlled state

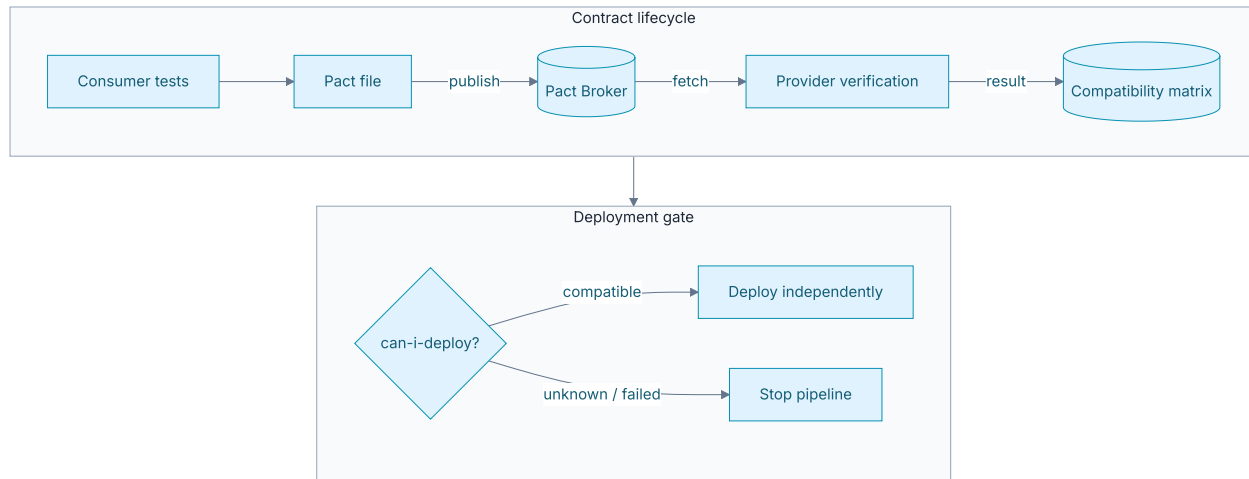
Dữ liệu test có thể tái tạo

Precise diff

Mismatch chỉ rõ field bị gãy

¹ **Verifier** — Công cụ Pact chạy phía Provider, replay từng request trong contract.

Broker & Deployment Gate¹



Broker liên kết **version Consumer + version Provider + kết quả verification** → Compatibility matrix

¹ Deployment gate — Điểm kiểm soát quyết định có được deploy hay không.

Case Study: Product Service

Bộ contract (10 interactions)

Nhóm API	Số	Trạng thái
GET /products	2	Có dữ liệu + danh sách rỗng
GET /product/:id	2	Tồn tại + không tồn tại
POST /products	2	Tạo thành công + validation error
PUT /product/:id	2	Cập nhật + không tồn tại
DELETE /product/:id	2	Xóa thành công + không tồn tại

Tóm tắt coverage



Case Study: Kết quả chạy

Kết quả

10/10

Consumer interactions
pass



Provider verification pass

Đây là case study chứng minh contract test chạy được trong repo thật, không chỉ là hình minh họa khái niệm.

Lệnh chạy

```
# Consumer test
npm run test:pact --prefix .../consumer

# Provider verification
npm run test:pact --prefix .../provider
```

Consumer: **FrontendWebsite**. Provider: **ProductService**.

Breaking Change Demo

Kịch bản

- Provider đổi field `name` → `title` trong response
- HTTP status vẫn **200** — API "có vẻ" hoạt động
- Nhưng Consumer contract yêu cầu field `name`

Kết quả

- Provider verification **FAIL** — thiếu field `name`
- Pact chỉ rõ mismatch: expected `name`, got `title`
- Khôi phục `name` → verification **PASS** trở lại

BÀI HỌC

Đổi tên field là **breaking change**¹ dù HTTP status không đổi.

Functional test có thể **không phát hiện** nếu không assert đúng field.

Contract test phát hiện **ngay tại provider verification**.

¹ **Breaking change** — Thay đổi làm Consumer cũ bị gãy (vd: đổi tên field).

PHẦN E

AI hỗ trợ kiểm thử

AI giúp tăng tốc sinh test, rà soát contract và gợi ý quy trình, nhưng mọi kết quả vẫn phải đi qua human review.

TOOLS

Postman, Cursor, Cline, Keploy

AGENT SKILL

Đóng gói prompt thành quy trình tái sử dụng

REVIEW

AI hỗ trợ, con người chịu trách nhiệm

AI trong quy trình Testing

Vai trò AI

- Sinh test case từ **API specification**
- Gợi ý cấu trúc **Postman Collection**
- Review contract, phát hiện **thiếu sót**
- Tạo sơ đồ, tài liệu và **workflow mẫu**

Công cụ đã khảo sát & áp dụng

- **Claude, ChatGPT, Gemini** — research & drafting
- **Postman Postbot** — sinh test script tự động
- **Cursor, Keploy, Kusho AI** — khảo sát & benchmark
- **Agent Skill** — đóng gói workflow sinh test

¹ **API specification** — Tài liệu mô tả cấu trúc API (endpoint, tham số, response).

² **Agent Skill** — Tập hướng dẫn đóng gói để AI tự động sinh test.

Nguyên tắc & Quy trình AI-First

1. Hướng dẫn từng bước

Giao việc có ngữ cảnh rõ ràng, tránh prompt chung chung.

2. Human Review 100%

Con người kiểm tra tính đúng đắn và an toàn trước khi dùng.

3. Min bẫy AI Audit Log

Ghi lại minh chứng, prompt và kết quả vào AI Audit Report.

4. Quality over Completion

Ưu tiên chất lượng và khả năng bảo trì hơn số lượng.

Trong seminar, AI được xem như cộng tác viên tạo bản nháp và gợi ý nhanh. Con người vẫn chịu trách nhiệm kiểm tra correctness, security và khả năng tái sử dụng.

Agent Skill — AI-driven Test Generator

INPUT

API Specification
(OpenAPI¹ / Markdown)

PROCESS

Phân tích endpoint →
Sinh test cases →
Tạo Postman Collection

OUTPUT

Collection JSON +
Data files +
Test scripts

Tính tái sử dụng

>80%

Mã nguồn/prompt tái sử dụng

100%

Agent Skill, workflows, Newman runner

Demo: Chạy Agent Skill trên Swagger PetStore API → chứng minh reusability

¹ OpenAPI — Chuẩn mô tả API phổ biến (file YAML/JSON).

PHẦN F

Tổng kết

Khép lại seminar bằng các nguyên tắc áp dụng, cách chọn đúng lớp test và những tài nguyên để người xem có thể tiếp tục tự học.

API TESTING

Kiểm tra chức năng, auth và dữ liệu

CONTRACT TESTING

Bảo vệ compatibility giữa Consumer và Provider

TAKEAWAY

Chọn đúng lớp test cho đúng rủi ro

API Testing vs Contract Testing

	API Testing	Contract Testing
Câu hỏi	Endpoint hoạt động đúng?	Consumer & Provider còn tương thích?
Công cụ	Postman + Newman	Pact
Phạm vi	Nhiều endpoint, nhiều case	Một cặp Consumer–Provider
Data	Data-driven (CSV/JSON)	Pact interactions + matchers
Automation	Newman trong GitHub Actions	Pact verification + can-i-deploy
Phát hiện	Lỗi chức năng, validation, auth	Breaking change tại biên
Bổ sung	✓ Cả hai cùng cần thiết	

Khi nào dùng gì?

Dùng API Testing khi

- Kiểm tra **chức năng** endpoint
- **Validation** input/output
- **Authentication** & Authorization
- **Regression suite**¹ cho API

Dùng Contract Testing khi

- Nhiều service phát triển **độc lập**
- Hay có **breaking change** khi tích hợp
- Cần **deployment gate** (can-i-deploy)
- Muốn **feedback sớm** trước staging

Chiến lược tối ưu: Unit → logic · Contract → compatibility · API/Integration → chức năng & wiring · Ít E2E → critical journeys

¹ **Regression suite** — Bộ test chạy lại mỗi khi có thay đổi để phát hiện lỗi cũ tái phát.

Adoption Path — Bắt đầu thế nào?

1 Chọn một ranh giới rủi ro
Cặp Consumer–Provider hay thay đổi/hay gây

2 Contract 1–2 luồng quan trọng
Success + một error behavior

3 Chạy trong CI
Consumer test + provider verification

4 Thêm Broker + can-i-deploy
Khi workflow ổn định

5 Đo hiệu quả
Lỗi phát hiện trước staging, thời gian feedback, số deploy bị chặn đúng

3 từ khóa

Fast

Feedback sớm tại local và CI

Focused

Khoanh đúng ranh giới Consumer-Provider

Safe

Các service tiến hóa độc lập có kiểm soát

API Testing + Contract Testing giải quyết hai nhóm rủi ro khác nhau.
Newman + GitHub Actions = regression suite tự động.
Pact = contract artifact + provider verification + compatibility gate.

Deliverables

Videos

- **Video 1:** Lý thuyết & Thuật ngữ API / Contract Testing
- **Video 2:** Hướng dẫn cài đặt môi trường
- **Video 3:** Demo thực hành (Postman + Newman + Pact & AI)

Tài liệu thực hành

- **Activity Worksheet** — 90 phút tại lớp
- **Mini Exercise** — Từ API Test đến Contract Breaking Change

Tóm tắt sử dụng

Cho người học

Có video, worksheet và mini exercise để tự luyện lại sau buổi seminar.

Cho người trình bày

Có checklist để dẫn lại phần demo, workflow và tài liệu tham khảo.

Repository & Official Docs

Repository

- [src/sample-api/pact-workshop-js/](#) — Pact source code
- [src/postman/](#) — Collections + data files
- [src/newman/](#) — Runner scripts
- [.github/workflows/](#) — CI/CD pipelines
- [docs/reports/week08/AI Usage/](#) — AI Audit Report + prompt logs

Tài liệu chính thức

- [docs.pact.io](#) — Pact Foundation
- [learning.postman.com](#) — Postman
- [github.com/postmanlabs/newman](#) — Newman

Toàn bộ deliverables và link chính thức đều được gom ở đây để dễ tra cứu sau buổi seminar.

Q&A & DISCUSSION

Hỏi & Đáp — Thảo luận

Seminar Kiểm thử Phần mềm — API Testing & Contract Testing

Mời Thầy/Cô & Các Bạn đặt câu hỏi 

THANK YOU

Chân Thành Cảm Ơn!

Cảm ơn Thầy/Cô và các bạn đã lắng nghe bài thuyết trình
của **Nhóm 03 — SEBros.**

Ngô Nguyễn Thế Khoa

23127065

Mạch Quốc Tấn

23127115

Ân Tiến Nguyên An

23127148

Nguyễn Tuấn Anh

23127152

Nguyễn Lê Hồ Anh Khoa

23127211